

HO CHI MINH CITY UNIVERSITY OF TRANSPORT (UTH)
FACULTY OF INFORMATION TECHNOLOGY

Capstone Project Document

Horse Racing Tournament Management System

Group Members	Ha Duy Hung - 074206002060 (<i>Team Leader</i>) Dinh Thuy Anh - 068306011497 Ho Nguyen Thai Chau - 054206003685 Le Thi My Hue - 052306012275 Bui Le Quang Huy - 231206000064 Nguyen Gia Huy - 2251120017 Doan Thi Thu May - 068306004742
Supervisor	Lecturer Nguyen Van Chien
Capstone Project code	Hourse Racing Tournament

Contents

Acknowledgement	3
Definition and Acronyms	4
1 Project Introduction	5
1.1 Overview	5
1.1.1 Project Information	5
1.1.2 Project Team	5
1.2 Product Background	5
1.3 Existing Systems	6
1.3.1 Prism Horse	6
1.3.2 RaceDB	7
1.4 Business Opportunity	8
1.5 Software Product Vision	8
1.6 Project Scope & Limitations	8
1.6.1 Major Features	8
1.6.2 Limitations & Exclusions	9
2 Software Project Management Plan	10
2.1 Overview	10
2.2 Scope & Estimation	10
2.3 Project Objectives	11
2.3.1 Defect Tolerance by Testing Phase	12
2.4 Risk Management	13
2.5 Management Approach	14
2.5.1 Agile-Scrum Implementation	14
2.5.2 Coding Standards and Bug Prevention	15
2.6 Deliverables Schedule	15
2.6.1 Deliverables Timeline (Week 1 - Week 9)	15
2.7 Responsibility Assignment Matrix (DRSI)	16
2.7.1 Document Management	17
2.7.2 Source Code Management	18
2.7.3 Tools and Infrastructure	18
3 Software Requirement Specification	20
3.1 Product Overview	20
3.2 User Requirements	20
3.3 Functional Requirements	20
3.3.1 Horse Owner Screen Flow	20

3.3.2	Horse Owner Screen description	21
3.4	Web Application	21
3.4.1	Horse Owner Login	21
3.4.2	Manage Horses	22
3.4.3	Invite Jockey	23
3.4.4	Register Tournament	24
3.4.5	Registered Tournaments	25
3.4.6	Horse Schedule	26
3.4.7	Race Results	27
3.4.8	Owner Profile	28
3.4.9	Ranking	29
3.4.10	Spectator Login	30
3.4.11	Prediction Management	31
3.4.12	View Schedule and Results	32
3.4.13	Personal Profile	33
3.4.14	Leaderboard	34
3.4.15	Admin Login	35
3.4.16	System Overview	36
3.4.17	Manage Tournaments	37
3.4.18	Schedule Races	38
3.4.19	Manage Rewards	39
3.4.20	User Management	40
3.4.21	Confirm Registrations	41
3.4.22	Leaderboard	42
3.5	Non-Functional Requirements	43
3.5.1	External Interfaces	43
3.5.2	Quality Attributes	44
4	Software Design Description	46
4.1	Overall Design	46
4.2	Detailed Design	47
4.2.1	Admin	47
4.2.2	Horse Owner Login and Registration	47
4.2.3	Jockey	49
4.2.4	Spectator	49
4.2.5	Referee	49

Acknowledgement

We would like to express our sincere gratitude to the faculty of Ho Chi Minh City University of Transport (UTH), and especially to our supervisor, Nguyen Van Chien, for their dedicated guidance and support throughout the development of this Capstone Project.

We would also like to thank every member of our team for their effort and close collaboration, which made it possible to complete the "Horse Racing Tournament Management System" project on schedule.

Definition and Acronyms

Term / Abbreviation	Definition
MVP	Minimum Viable Product
SRS	Software Requirement Specification
SDD	Software Design Description
ERD	Entity Relationship Diagram
UC	Use Case
BR	Business Rule
CR	Common Requirement
MSG	Application Message
UI	User Interface
API	Application Programming Interface
DB	Database

I. Project Introduction

1.1. Overview

1.1.1 Project Information

- **Project name:** Horse Racing Tournament Management System
- **Project code:**
- **Group name:** Group 3
- **Software Type:** Web application

1.1.2 Project Team

Full Name	Role	Email	Mobile
Nguyen Van Chien	Supervisor	chiennv@ut.edu.vn	
Ha Duy Hung	Leader	haduydung0912@gmail.com	
Dinh Thuy Anh	Member	anhdt1497@ut.edu.vn	
Ho Nguyen Thai Chau	Member	chauhnt3685@ut.edu.vn	
Le Thi My Hue	Member	lemyhue1803@gmail.com	
Nguyen Gia Huy	Member	nhuy22174@gmail.com	
Doan Thi Thu May	Member	maydtt6742@ut.edu.vn	
Bui Le Quang Huy	Member	huyblq0064@ut.edu.vn	

Table 1.1: Project Team

1.2. Product Background

In the context of Vietnam’s growing interest in sports entertainment and digital transformation, horse racing organizations are increasingly facing the need to manage tournaments, participants, horses, jockeys, and race registration processes more efficiently. However, many racing-related activities may still depend on manual management methods such as paper forms, scattered Excel files, direct communication, or disconnected systems. This can lead to difficulties in managing tournament information, horse and jockey profiles, owner registrations, approval status, and communication between horse owners, jockeys, referees, and administrators.

Recognizing these limitations, a student team has proposed the development of a Horse Racing Management System. This system aims to digitize the main workflow of horse racing event management, from tournament creation and participant registration to approval, race organization, and result tracking. The target users include horse owners who register horses for tournaments, jockeys who participate in races, administrators who manage tournaments and registration requests, referees who monitor race results, and viewers or related users who need to follow tournament information in a transparent and convenient way.

1.3. Existing Systems

1.3.1 Prism Horse

Link: <https://prism.horse>

Description: A horse racing management platform supporting race events, horse information, and participant management.

Actors:

- Horse Owners
- Jockeys
- Administrators
- Referees

Features:

- Horse and jockey management.
- Tournament organization.
- Race registration.
- Participant management.

Pros:

- Designed specifically for horse racing activities.
- Supports race and participant management.

Cons:

- Limited owner–jockey collaboration workflow.
- No invitation management mechanism.
- Limited registration status tracking.
- Lack of integrated communication features.

1.3.2 RaceDB

Link: <https://www.racedb.com>

Description: A race event management system that supports competition registration, participant management, race scheduling, and result tracking. The platform is primarily designed for cycling and sporting competitions.

Actors:

- Participants
- Event Organizers
- Administrators
- Race Officials

Features:

- Online participant registration.
- Race scheduling and event management.
- Result recording and ranking.
- Participant and competition tracking.

Pros:

- Supports online registration and race management.
- Provides race scheduling and result tracking.
- Offers a centralized system for competition administration.

Cons:

- Not specifically designed for horse racing.
- Does not support horse owner, horse, and jockey relationships.
- Lacks invitation and approval workflows between horse owners and jockeys.
- Does not provide horse profile management and ownership information.
- Limited support for horse racing officials and tournament-specific workflows.

1.4. Business Opportunity

The horse racing industry is gradually embracing digital transformation as racing organizations and clubs seek more efficient ways to manage tournaments, participants, and race operations. However, many horse racing activities still rely on manual registration processes, paper documents, and fragmented communication between horse owners, jockeys, and organizers. This creates an opportunity to develop a specialized management system that can streamline tournament registration, improve participant coordination, and enhance transparency throughout the racing process.

The proposed Horse Racing Management System aims to optimize race organization by supporting online tournament registration, horse and jockey management, invitation workflows, and registration approval processes. The system can reduce administrative workload, minimize registration errors, and improve communication among stakeholders.

The system will compete in the sports event management and competition management software market while specifically targeting the horse racing domain, where existing solutions often lack comprehensive support for horse owner-jockey collaboration, race registration workflows, and tournament management requirements. By focusing on the unique characteristics of horse racing activities, the system has the potential to provide a more specialized and efficient solution for racing organizations and participants.

1.5. Software Product Vision

For horse racing organizations seeking to digitalize tournament operations, the Horse Racing Management System is a web-based platform that enables horse owners to register horses, invite jockeys, and participate in racing tournaments through an integrated online process. Jockeys can manage invitations and confirm participation, while administrators and referees can efficiently monitor registrations, approve requests, and oversee race activities.

Unlike traditional management methods that rely on paper documents, manual registration, and fragmented communication, the Horse Racing Management System reduces administrative workload, improves the accuracy and transparency of tournament management, and enhances collaboration among horse owners, jockeys, referees, and administrators. By digitalizing registration workflows, participant information, and competition management, the system also establishes a foundation for future services such as performance analytics, race statistics, online event management, and data-driven decision making in horse racing organizations.

1.6. Project Scope & Limitations

1.6.1 Major Features

Major Features for Admin

FE-01: Login/Logout

FE-02: Manage user accounts

Major Features for Horse Owner

FE-01: Register horse for tournament

FE-02: View registration status

1.6.2 Limitations & Exclusions

LI-1: Write limitation here.

II. Software Project Management Plan

This chapter describes project planning, scheduling, risk management, communication and configuration management.

2.1. Overview

The Software Project Management Plan (SPMP) outlines the managerial, technical, and resource allocation strategies for the development of the Horse Racing Tournament Management System. This project aims to digitize and automate the processes of managing horse racing tournaments, including horse owner registration, jockey invitation workflows, race scheduling, referee assignment, result recording, and spectator interactions.

The primary objective of this plan is to ensure that the project is delivered on time, within scope, and meets all defined quality criteria. To achieve these goals, the team employs an Agile-Scrum methodology, which promotes iterative development, continuous feedback, and rapid response to changes. This document details the project scope, estimation, quality objectives, risk management, and configuration management processes that govern the lifecycle of this project.

2.2. Scope & Estimation

To ensure the project is delivered successfully within the constraints of time and resources, the project scope has been broken down into a detailed Work Breakdown Structure (WBS). The WBS divides the Horse Racing Tournament Management System into 7 distinct modules, covering administrative tasks, core infrastructure, and functional components for various user roles.

Each work item is assessed for complexity (Simple, Medium, or Complex) based on the database interactions, business logic density, and interface requirements:

- **Simple:** Basic CRUD operations, simple UI layouts, or minor configurations requiring 5 to 6 man-days.
- **Medium:** Standard functional features with moderate business logic (e.g., forms validation, standard data list with filters) requiring 7 to 8 man-days.
- **Complex:** Critical components involving advanced logic, multiple integrations, or strict constraints (e.g., conflict checking algorithms, two-step referee verification, real-time leaderboard calculations) requiring 9 to 15 man-days.

The table below outlines the estimated effort in man-days for each WBS item based on these parameters:

No.	WBS Item	Complexity	Est. Effort (man-days)
1	PM01. Project Management & Administration		20
1.1	Project Setup & Git Workflow Config	Simple	5
1.2	Code Review, Integration & Conflict Support	Complex	15
2	DB01. Database & Infrastructure		12
2.1	Database Schema Setup & ERD Design	Medium	7
2.2	API Core Config & DB Connection	Simple	5
3	FE01. Authentication & User Management		14
3.1	Login, Registration & Authorization UI/API	Medium	8
3.2	User Management Panel (Pagination/Search)	Medium	6
4	FE02. Horse & Jockey Management		23
4.1	Horse Owner Profile Update & Registration	Medium	8
4.2	Jockey Profile Update & Invitation UI/API	Complex	15
5	FE03. Tournament & Race Scheduling		23
5.1	Tournament Management & Race Creation	Medium	8
5.2	Race Scheduling & Conflict Validation	Complex	15
6	FE04. Race Execution & Results		24
6.1	Referee Two-step Result Submission	Complex	12
6.2	Real-time Leaderboard & Prize Config	Complex	12
7	FE05. Spectator Portal		14
7.1	Schedules & Live Results UI/API	Simple	6
7.2	Place Prediction Form & Timer Lock	Complex	8

Table 2.1: Work Breakdown Structure (WBS) and Effort Estimation

2.3. Project Objectives

To ensure the Horse Racing Tournament Management System meets the quality standards required for deployment and user satisfaction, the team has established specific quality objectives. These objectives focus on functional correctness, performance, security, data integrity, and usability, matching the real development and testing guidelines of the project.

The table below outlines the core quality objectives and target criteria for the project:

No.	Quality Objective	Target Metric / Criteria	Measurement Method
1	Functional Suitability	100% of defined SRS requirements pass manual E2E test cases	Execution of Manual E2E Test Cases (Phase 1, 2, 3, 4)
2	Data Integrity & Encoding	100% successful storage and display of Vietnamese characters (Unicode) in DB and UI	Database inspection (SQL Server) & UI display validation
3	Security & Authentication	Secure password hashing (bcrypt) and JWT authorization for protected role-based routes	Manual code review of app security handlers and route guards
4	Performance Efficiency	Response time < 2s for all frontend page transitions and backend API calls	Network monitoring via Browser Developer Tools
5	Usability & Aesthetics	Responsive dark-mode glass-morphic interface with consistent date/time formats	Manual peer testing & UI checklist validation

Table 2.2: Project Quality Objectives

2.3.1 Defect Tolerance by Testing Phase

Defect density and resolution rates are tracked continuously across various testing levels: Document Review, Integration Testing (Phase 1-2), Integration Testing (Phase 3), and System Testing/UAT. Defects are classified into three severity levels:

- **Critical:** System crash, database connection failure, or core functions completely blocked (e.g., authentication failure, scheduling conflicts, failed result recalculation).
- **Major:** Features malfunctioning or displaying incorrect data without completely blocking the flow (e.g., jockey showing ID instead of real name, date formats displaying raw ISO string).
- **Minor:** UI layout alignment bugs, minor text formatting glitches, or spelling errors that do not affect functional operations.

The table below details the maximum allowed defects and the target resolution process for each testing phase:

Testing Phase	Max Critical	Max Major	Max Minor	Resolution Process
Document Review	0	Max 2	Max 5	Resolved before coding phase
Unit Testing	0	0	Max 3	Resolved before PR merge
Integration Test	0	Max 1	Max 8	Resolved before System Test
System Test	0	Max 2	Max 12	Resolved before UAT phase
Acceptance Test (UAT)	0	0	Max 3	Resolved before final release

Table 2.3: Maximum Allowed Defect Rates by Testing Phase

2.4. Risk Management

To ensure the smooth execution of the Horse Racing Tournament Management System development, the team has identified potential technical, operational, and managerial risks. Managing these risks involves analyzing their probability of occurrence, potential impact, and establishing proactive mitigation and contingency plans.

The table below presents the key risks identified during the project lifecycle and the corresponding response strategies:

No.	Risk Description	Probability	Impact	Mitigation / Response Plan
1	Git Merge Conflicts: Code integration conflicts due to multiple developers working on adjacent modules (e.g., dashboard panels).	High	Major	Split large files (like <code>dashboard/page.js</code>) into smaller components. Enforce strict branching guidelines: pull changes from <code>dev-GiaHuy</code> before coding, and create pull requests for review.
2	Vietnamese Encoding Errors: Vietnamese names and text showing as ? in UI or database due to non-Unicode configurations in SQL Server.	Medium	Moderate	Change string definitions to <code>Unicode</code> and text to <code>UnicodeText</code> in SQLAlchemy models (<code>database_models.py</code>) to map to <code>NVARCHAR</code> columns in SQL Server.
3	Race Scheduling Conflicts: Double-booking of referees, jockeys, or horses in overlapping races or tournaments.	Medium	Major	Implement robust backend validations (<code>Conflict Validation</code>) for schedule updates and referee assignments before persisting to the database.
4	React Hooks Rule Violations: Frontend crashes and rendering bugs caused by calling Hooks inside loops or conditional statements (e.g., in Admin panels).	Medium	Major	Perform mandatory code reviews and utilize ESLint static analysis to identify Hook violations prior to code integration.
5	Inconsistent Local Databases: Out-of-sync local database schemas among team members causing API integration failures.	High	Major	Provide a centralized database setup and seeding script (<code>db_setup.py</code>). Enforce running the script immediately after git pulls.

Table 2.4: Project Risk Analysis and Mitigation Plans

2.5. Management Approach

To effectively manage the development of the Horse Racing Tournament Management System, the team employs the Agile-Scrum methodology. This approach allows the team to deliver features iteratively, adapt to changes quickly, and ensure constant communication among members.

2.5.1 Agile-Scrum Implementation

The development cycle is organized into weekly sprints. Each sprint begins with a planning meeting and ends with a review and retrospective.

- **Sprint Planning:** Held at the start of each week to define the sprint goal and allocate tasks from the product backlog on Jira.
- **Weekly Syncs:** Held twice a week (on Wednesdays and Saturdays) to track progress, identify blockers, and align integration efforts. Daily updates and progress reports are shared via Zalo.
- **Sprint Review & Retrospective:** Conducted at the end of the sprint to review completed tasks, demo features, and reflect on improvements for the next sprint.

2.5.2 Coding Standards and Bug Prevention

To maintain code quality and prevent bugs early in the lifecycle, the team enforces the following rules and standards:

- **Coding Style Guidelines:** The backend follows PEP 8 guidelines for Python (FastAPI), while the frontend adheres to ESLint standards for React/Next.js. React Hooks rules are strictly monitored (e.g., avoiding calling hooks inside loops or conditions).
- **Database Consistency:** Any database schema modifications must be updated in the SQLAlchemy models. Developers are required to run the automated database setup and seeding script (`db_setup.py`) locally after pulling the latest code to prevent database mismatches.
- **Git Workflow Rules:** Direct pushes to the main branch (`main`) or integration branch (`dev-GiaHuy`) are prohibited. All work must be completed on feature branches (`feature/*`) and integrated via Pull Requests (PRs). The Team Leader reviews and approves all PRs before merging.

2.6. Deliverables Schedule

The project deliverables are scheduled and tracked weekly, aligning with the Agile sprints and the development phases of the Horse Racing Tournament Management System. The following schedule details the deliverables for both source code components (API and UI) and project documentation from Week 1 to Week 9.

2.6.1 Deliverables Timeline (Week 1 - Week 9)

The timeline below structures the actual development history, pull requests, and documentation milestones into weekly progress updates from the beginning of the project through the final LaTeX compilation:

Week	Phase / Focus	Key Deliverables (Source Code & Documentation)	Status
Week 1	Scope & Context	System Scope definition, Actors identification, Context Diagram, and initial database setup (<code>schema.sql</code>).	Completed
Week 2	Requirements	Overall Use Case Diagram for MVP, Use Case Descriptions, and initial project layout setup.	Completed
Week 3	Phase 1 (Core Auth)	Next.js and FastAPI project skeleton, user registration, login flow, and dashboard layout restructuring.	Completed
Week 4	Phase 1 (Admin Flow)	User Management API, lock/unlock accounts, dynamically calling referees list, and fixing React Hooks rendering violations. Merged PR #4 and #6 (16/06/2026).	Completed
Week 5	Phase 2 (Owner & Jockey)	Jockey invitation workflow (Accept/Reject), horse profiles CRUD with age validation (2-10 years), and registration status updates. Merged (19/06/2026).	Completed
Week 6	Phase 3 (Referee & Spectator)	Two-step result verification (<code>RESULTS_ENTERED</code> → <code>COMPLETED</code>), <code>RaceInspections</code> form, spectator place prediction form with dynamic filters, and auto-reward points logic. Merged (21/06/2026).	Completed
Week 7	Phase 4 (Admin & Database)	Timezone <code>Asia/Ho_Chi_Minh</code> integration, $\pm 2h$ referee conflict check, validation rules, database schema update (missing columns for users, spectators), and fixing local connections (127.0.0.1). Merged (24/06/2026).	Completed
Week 8	LaTeX	LaTeX compilation environment setup, project structure creation, and initial drafts of Chapter 1 and 2 sections.	Completed
Week 9	LaTeX Compilation	Finalizing Chapter 2 (SPMP) sections, Chapter 3 (SRS) requirements, Chapter 4 (SDD) system design, and compiling the official PDF document.	Completed

Table 2.5: Key Deliverables Schedule (Week 1 - 9)

2.7. Responsibility Assignment Matrix (DRSI)

To clarify roles and responsibilities across the team, the project utilizes a DRSI matrix (Do, Review, Support, Informed). The abbreviations used in the matrix are defined as follows:

- **D (Do):** The member responsible for developing and implementing the deliverable.

- **R (Review)**: The member responsible for reviewing and approving the quality of the work.
- **S (Support)**: The member providing support during development or integration.
- **I (Informed)**: Members kept updated on the status of the deliverable.
- **- (Omitted)**: Member has no direct responsibility for the deliverable.

The table below outlines the DRSI matrix for the key project deliverables:

Deliverable / Module	DH	TA	GH	TC	MH	BH	TM
Project Setup & Configuration	D	I	I	I	I	I	I
Database & Infrastructure Setup	R	I	D	I	I	S	I
Authentication & User Management	R	I	D	I	D	I	I
Horse Owner Registration	R	D	I	I	I	I	I
Jockey Profile & Invitations	R	S	I	D	I	I	I
Race Scheduling & Referee Assign	R	I	S	I	D	I	I
Race Result & Ranking Calculation	R	I	I	I	I	D	I
Spectator Prediction & Rewards	R	I	I	I	I	S	D
LaTeX Report & Presentation	D	D	D	D	D	D	D

Table 2.6: DRSI Responsibility Assignment Matrix

2.7.1 Document Management

To ensure consistent document formatting and prevent conflicts during collaborative writing, the team follows a structured LaTeX document management workflow on GitHub:

- **Three-Tier Architecture**: The document structure is divided into three tiers:
 1. `main.tex`: The root file that only handles package imports and includes document wrappers.
 2. `documents/`: Wrapper files for each chapter (e.g., `project_management_plan.tex`).
 3. `sections/`: The folder containing the actual content files divided by section and assignee.
- **Conflict Avoidance**: Team members are strictly restricted to editing files under their assigned sections. Editing `main.tex` or files in the `documents/` folder is prohibited without prior team alignment.
- **Figure Convention**: Figures must be stored in the `figures/` directory and named according to the format: `<chapter>_<content>.<ext>` (e.g., `figures/srs_erd.png`). Generic names like `image1.png` are avoided.

2.7.2 Source Code Management

The project source code is hosted on GitHub and managed using a customized feature-branching workflow to guarantee code quality and repository stability:

- **Branching Strategy:**

- **main:** Reserved for the final, production-ready release used for demos.
- **dev-GiaHuy:** The main integration branch where all validated feature branches are merged.
- **feature/*:** Temporary branches used by developers to work on assigned tasks (e.g., `feature/auth-user`, `feature/spectator-view`).

- **Development Protocol:**

1. Pull the latest updates from the remote integration branch before coding: `git pull origin dev-GiaHuy`.
2. Develop the feature on a separate `feature/*` branch.
3. Push the feature branch to the remote repository.
4. Open a Pull Request (PR) from the feature branch to `dev-GiaHuy`. Direct pushes to `dev-GiaHuy` or `main` are disabled.

- **Commit Message Convention:** Commit messages must clearly state the scope of the change. For example:

- `git commit -m "Add login API"`
- `git commit -m "Create horse management page"`

- **Review & Merge:** The Team Leader reviews all PRs, checks for build errors, resolves conflicts, and merges the code into the integration branch.

2.7.3 Tools and Infrastructure

The development and execution environment for the Horse Racing Tournament Management System utilizes the following tech stack, tools, and local infrastructure:

- **Programming Languages & Frameworks:**

- **Backend:** Python 3 utilizing the FastAPI framework for clean architecture REST APIs.
- **Frontend:** JavaScript/React.js utilizing Next.js (App Router) for the web application UI.

- **Database Management:**

- Local Microsoft SQL Server Express instance (`localhost\SQLEXPRESS`) with database named `HorseRacing`.
- Connection library: `pyodbc` (ODBC Driver 17 for SQL Server) and `SQLAlchemy` ORM.

- **Development Platforms:**
 - **Operating System:** Windows 10/11 workstation.
 - **Integrated Development Environment (IDE):** VS Code equipped with LaTeX Workshop extension.
 - **Git Client:** Git for Windows / Git Bash.
 - **LaTeX Compiler:** MiKTeX or TeX Live for generating report PDFs locally.
- **Local Hosting Configuration:**
 - Backend server running locally on `http://127.0.0.1:8000` (API Client configured directly to IP address to prevent connections dropouts).
 - Frontend Next.js development server running locally on `http://localhost:3000`.

III. Software Requirement Specification

3.1. Product Overview

3.2. User Requirements

3.3. Functional Requirements

3.3.1 Horse Owner Screen Flow

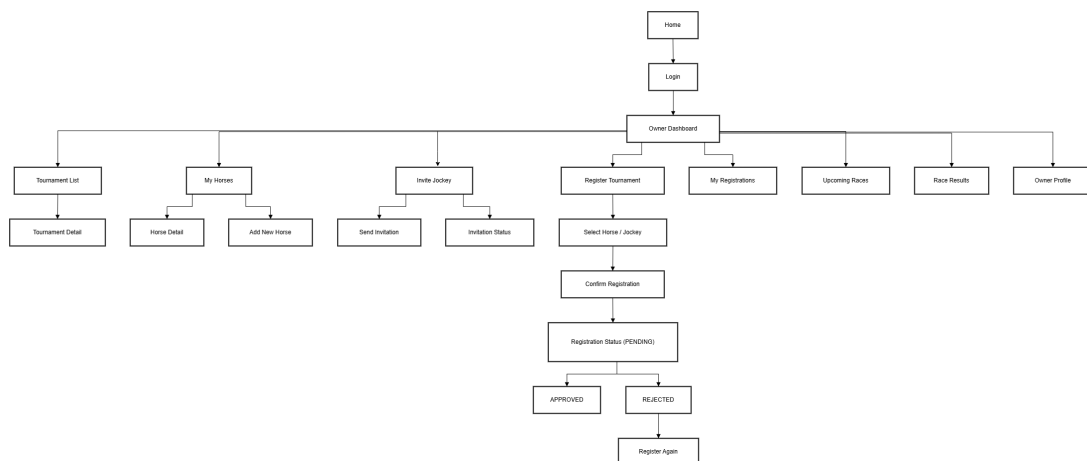


Figure 3.1: Horse Owner screen flow

3.3.2 Horse Owner Screen description

#	Feature	Screen	Description
1	General	Home	Landing page for all users; navigate to Login and Owner access.
2	General	Login	Authenticate the owner and enter the Owner Dashboard.
3	Dashboard	Owner Dashboard	Main hub for Owner with navigation to horse management, invitations, tournament registration, schedules, results, and profile.
4	Horse Management	My Horses	View and manage owned horses, including add, edit, and delete operations.
5	Horse Management	Horse Detail	Edit horse details: name, age (2-10), breed, gender, and save changes.
6	Invitation	Invite Jockey	Send invitations to jockeys for a selected horse and tournament with a custom message.
7	Invitation	Invitation Status	Track invitations with jockey, horse, tournament, and current acceptance status.
8	Registration	Register Tournament	Register accepted horse/jockey pairs for tournaments and create registration requests.
9	Registration	My Registrations	Display tournament registration history with approval status: APPROVED, PENDING, REJECTED.
10	Schedule	Upcoming Races	Show upcoming races for the owner's horses with date, time, and location.
11	Results	Race Results	Show historical race results for the owner's horses with rank, points, notes, and violations.
12	Profile	Owner Profile	View and update owner details: full name, phone, company, avatar.

Table 3.1: Horse Owner screen descriptions

3.4. Web Application

3.4.1 Horse Owner Login

Role: Horse Owner

- **Function trigger:** The owner enters account credentials to log in.

- **Function description:**

- **Purpose:**

- 1. Verify the owner's identity.
 - 2. Grant access to owner functionalities.

- **Data processing:**

- 1. The owner enters username and password.
 - 2. The system validates credentials.
 - 3. The system identifies the OWNER role.
 - 4. The system creates a login session.

- **Function details:**

- Successful login redirects to Owner Dashboard.
 - Invalid credentials display an error message.
 - Users without accounts may register.

Screen layout:

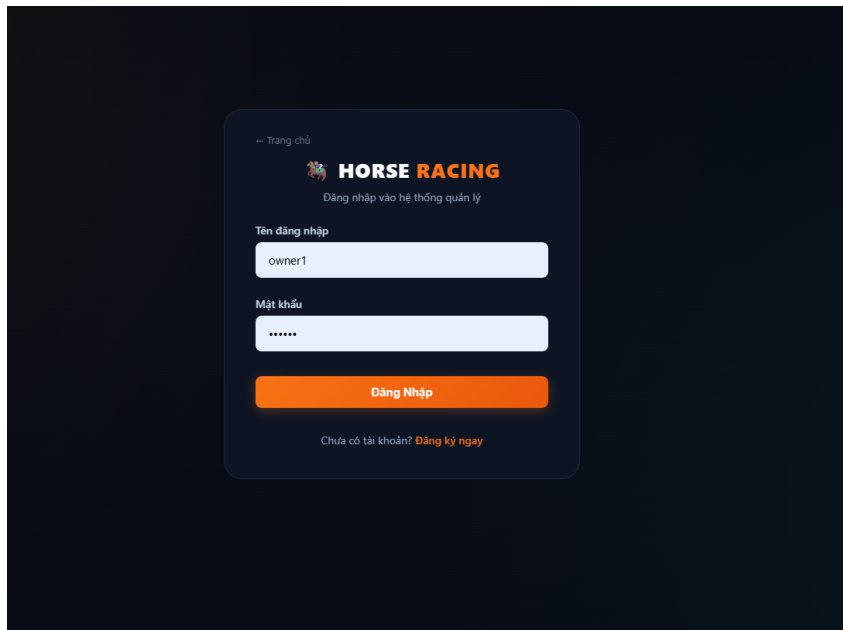


Figure 3.2: Horse Owner Login

3.4.2 Manage Horses

Role: Horse Owner

- **Function trigger:** Select “Quan ly Ngua”.

- **Function description:**

- **Purpose:**

- 1. Register new horses.

- 2. Manage owned horses.
- **Data processing:**
 1. Enter horse name.
 2. Enter age.
 3. Select breed.
 4. Select gender.
 5. Save horse information.
- **Function details:**
 - Add new horses.
 - Edit horse information.
 - Delete horse information.
 - Display horse list.

Screen layout:

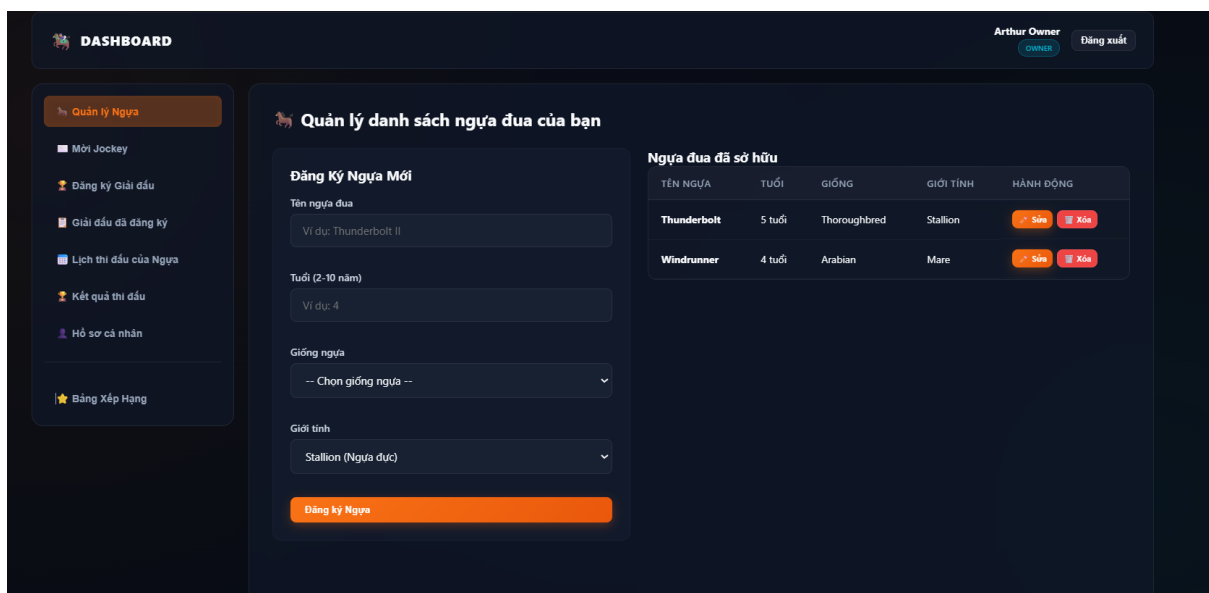


Figure 3.3: Manage Horses

3.4.3 Invite Jockey

Role: Horse Owner

- **Function trigger:** Select “Moi Jockey”.
- **Function description:**
 - **Purpose:**
 1. Invite jockeys.
 2. Create horse-jockey partnerships.

- **Data processing:**
 1. Select jockey.
 2. Select horse.
 3. Select tournament.
 4. Enter invitation message.
 5. Send invitation.
- **Function details:**
 - Invitation status is displayed.
 - Pending invitations can be monitored.
 - Accepted invitations may be used for registration.

Screen layout:

DASHBOARD Arthur Owner Đang xuất

Mời Jockey

Mời Jockey lái ngựa thi đấu

Gửi Lời Mời Mới

Chọn Jockey
-- Chọn Jockey --

Chọn Ngựa đua
-- Chọn Ngựa --

Chọn Giải đấu
-- Chọn Giải đấu --

Tin nhắn gửi kèm
Mời tham gia giải đua...

Gửi Lời Mời

Trạng thái lời mời đã gửi

JOCKEY	NGỰA	GIẢI ĐẤU	TRẠNG THÁI
Sarah Jockey	Windrunner	Summer Championship 2026	PENDING

Figure 3.4: Invite Jockey

3.4.4 Register Tournament

Role: Horse Owner

- **Function trigger:** Select “Dang ky Giai dau”.
- **Function description:**
 - **Purpose:**
 1. Register horse-jockey pairs.
 2. Participate in tournaments.
 - **Data processing:**

1. Retrieve available tournaments.
2. Check accepted jockey invitations.
3. Select horse-jockey pair.
4. Submit registration request.

- **Function details:**

- Only accepted invitations may register.
- Registration requests are sent to administrators.
- Invalid pairs cannot participate.

Screen layout:

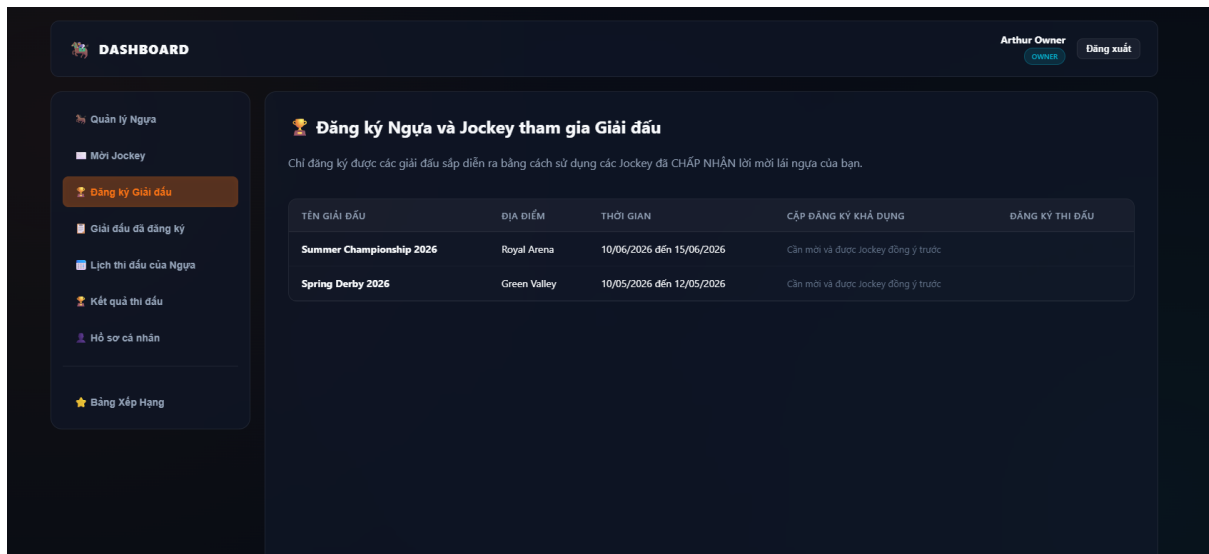


Figure 3.5: Tournament Registration

3.4.5 Registered Tournaments

Role: Horse Owner

- **Function trigger:** Select “Giai dau da dang ky”.
- **Function description:**
 - **Purpose:**
 1. Monitor registration status.
 2. View approval results.
 - **Data processing:**
 1. Retrieve registration records.
 2. Retrieve approval status.
 3. Display registration history.
- **Function details:**

- Display horse information.
- Display jockey information.
- Display approval status.

Screen layout:

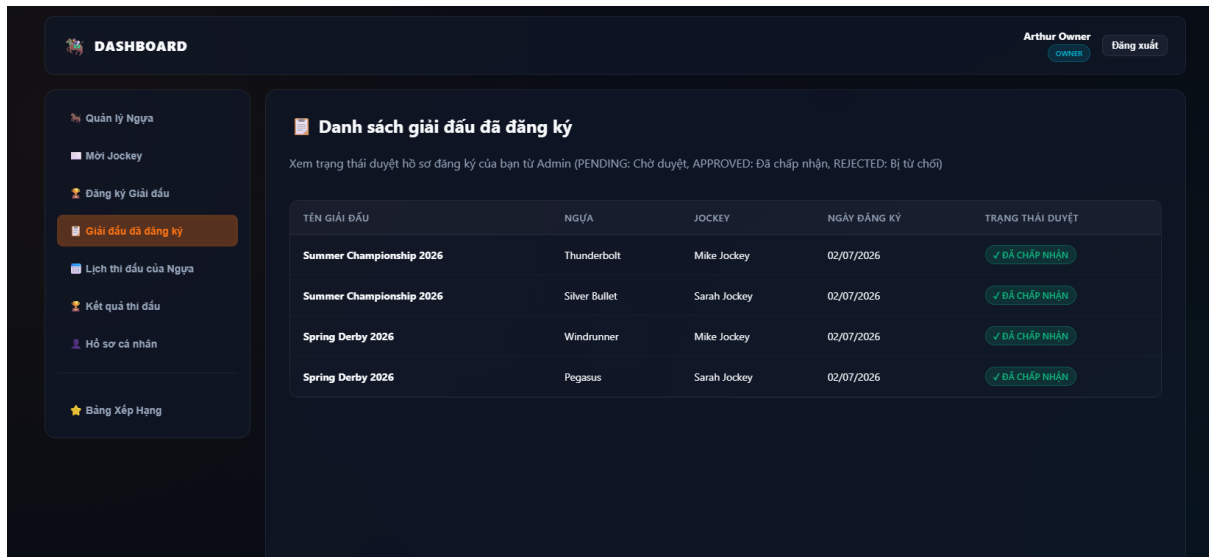


Figure 3.6: Registered Tournaments

3.4.6 Horse Schedule

Role: Horse Owner

- **Function trigger:** Select “Lịch thi đấu của Ngựa”.
- **Function description:**
 - **Purpose:**
 1. View upcoming races.
 2. Prepare for competitions.
 - **Data processing:**
 1. Retrieve race schedules.
 2. Retrieve tournament information.
 3. Display race information.
- **Function details:**
 - Display race date.
 - Display location.
 - Display tournament information.

Screen layout:

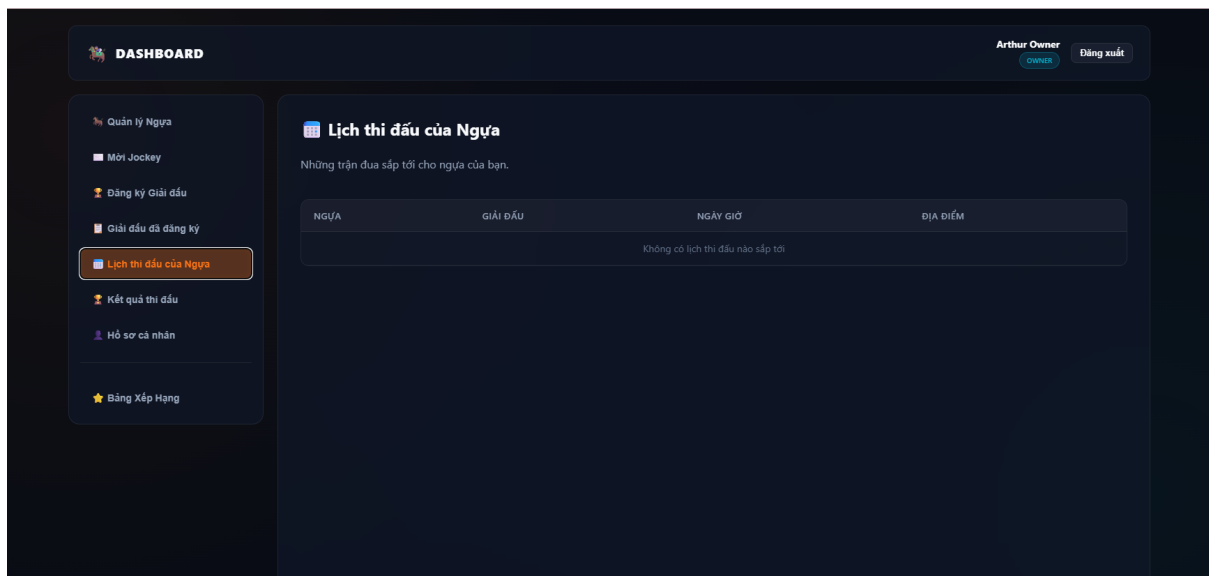


Figure 3.7: Horse Schedule

3.4.7 Race Results

Role: Horse Owner

- **Function trigger:** Select “Ket qua thi dau”.
- **Function description:**
 - **Purpose:**
 1. Review race results.
 2. Evaluate horse performance.
 - **Data processing:**
 1. Retrieve completed races.
 2. Retrieve rankings.
 3. Retrieve scores.
 4. Retrieve violations.
- **Function details:**
 - Display race rankings.
 - Display accumulated points.
 - Display remarks.
 - Display violations.

Screen layout:

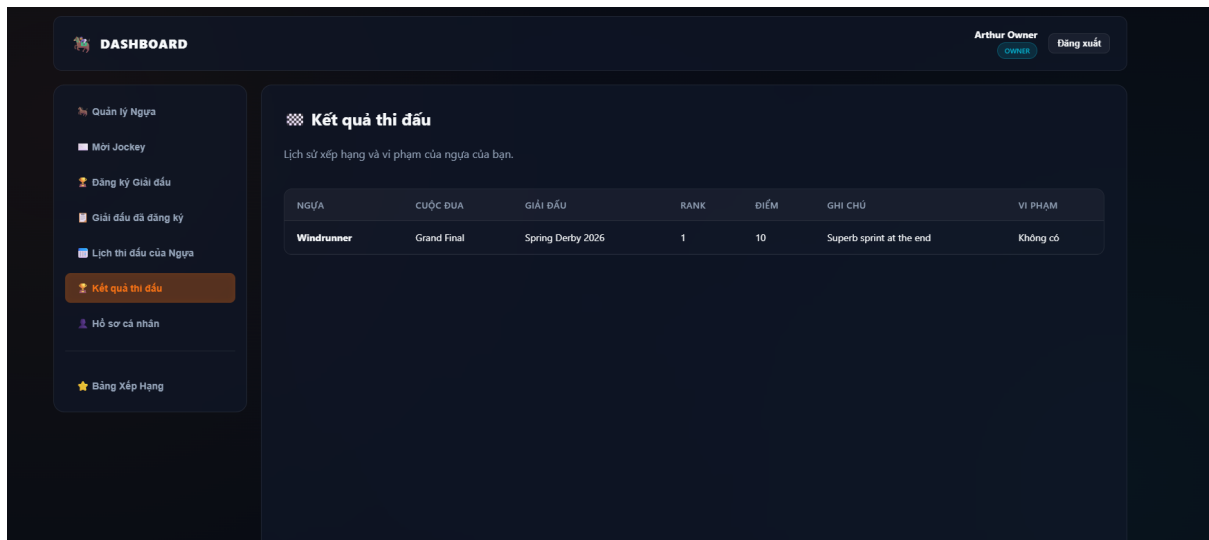


Figure 3.8: Race Results

3.4.8 Owner Profile

Role: Horse Owner

- **Function trigger:** Select “Ho so ca nhan”.
- **Function description:**
 - **Purpose:**
 1. Manage personal information.
 2. Update profile information.
 - **Data processing:**
 1. Update personal information.
 2. Update contact information.
 3. Update company information.
 4. Save profile data.
- **Function details:**
 - Edit owner profile.
 - Edit biography.
 - Update social information.

Screen layout:

DASHBOARD

Arthur Owner

Đăng xuất

Quản lý ngựa

Wish Jockey

Đăng ký Cầu đấu

Cầu đấu đã đăng ký

Lịch thi đấu của Ngựa

Kiểm tra thi đấu

Mô sơ cá nhân

Đăng Xếp hạng

Hồ sơ cá nhân Chủ Sở Hữu

Chỉnh sửa thông tin

Họ tên

Arthur Owner

Email

owner1@honoracing.com

Số điện thoại

Tuổi

Kinh nghiệm (năm)

Nghề nghiệp

Địa chỉ liên hệ

Quốc tịch

-- Chọn quốc tịch --

Công ty / Tên đội

Apex Racing Stables

Avatar (URL)

Website / Mạng xã hội

https://example.com

Mô tả ngắn (Bio)

Ngày tham gia hệ thống

Thành viên từ 02/07/2026

Lưu thay đổi

Thông tin hiện tại

Họ tên: Arthur Owner

Email: owner1@honoracing.com

Thành viên từ: 02/07/2026

SĐT: Chưa có

Tuổi: Chưa có

Kinh nghiệm: Chưa có

Nghề nghiệp: Chưa có

Địa chỉ: Chưa có

Quốc tịch: Chưa có

Avatar: Chưa có

Công ty: Apex Racing Stables

Website / Mạng xã hội: Chưa có

Bio: Chưa có

Figure 3.9: Owner Profile

3.4.9 Ranking

Role: Horse Owner

- **Function trigger:** Select “Bang xep hang”.
- **Function description:**
 - **Purpose:**
 1. View official rankings.
 2. Compare competition performance.
 - **Data processing:**

29

1. Retrieve horse rankings.
2. Retrieve jockey rankings.
3. Filter tournaments.

- **Function details:**

- Display ranking positions.
- Display accumulated scores.
- Display tournament filters.

Screen layout:

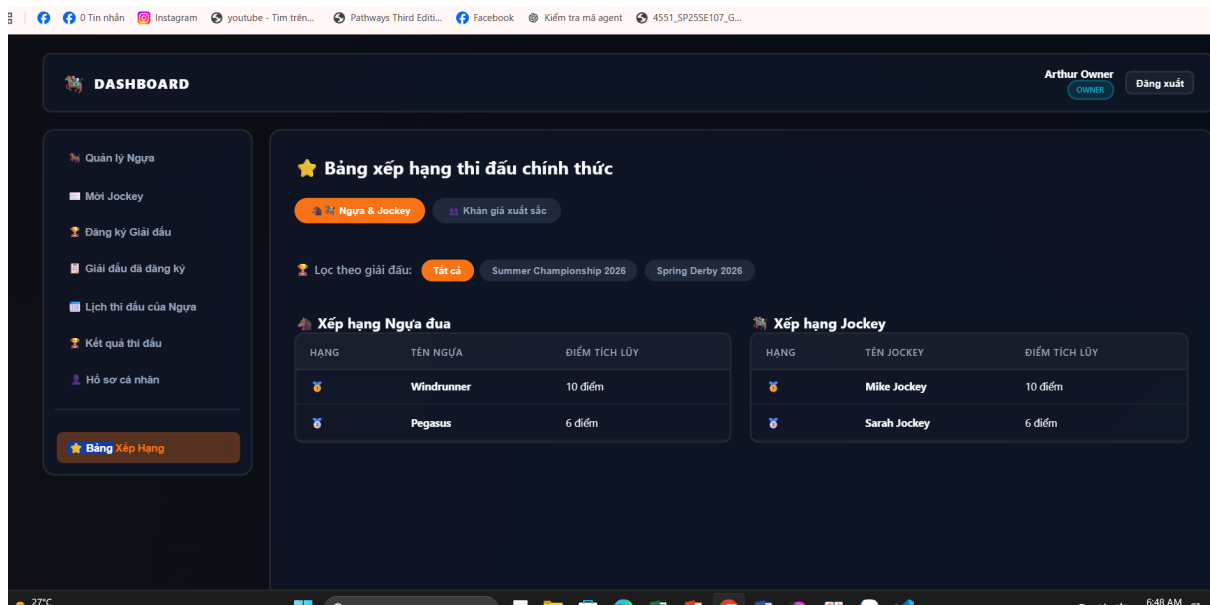


Figure 3.10: Ranking

3.4.10 Spectator Login

Role: Spectator

- **Function trigger:** The spectator enters account credentials to log in.
- **Function description:**
 - **Purpose:**
 1. Verify the spectator's identity.
 2. Grant access to spectator functionalities.
 - **Data processing:**
 1. The spectator enters username and password.
 2. The system validates credentials.
 3. The system identifies the SPECTATOR role.
 4. The system creates a login session.

Screen layout:

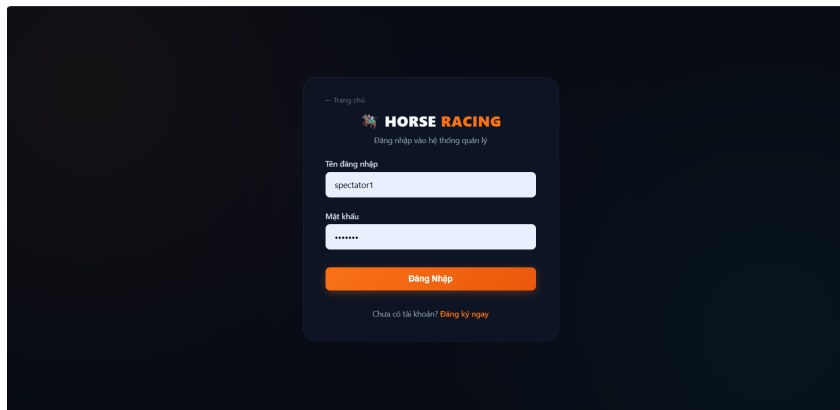


Figure 3.11: Spectator Login

3.4.11 Prediction Management

Role: Spectator

- **Function trigger:** The spectator navigates to the "Du doan Tran dua" (Predict Race) menu.
- **Function description:**
 - **Purpose:**
 1. Allow the spectator to submit a new race ranking prediction to earn reward points.
 2. Allow the spectator to view, edit, or delete their existing prediction history.
 - **Data processing:**
 1. **For creating a new prediction:**
 - * The spectator selects a race, selects a horse, and chooses the predicted finishing rank from the dropdown menus.
 - * The spectator clicks the "Gui du doan" button.
 - * The system validates the prediction data and saves it to the database.
 2. **For managing prediction history:**
 - * The system retrieves and displays the spectator's accumulated reward points and prediction history list (including Race, Horse, Predicted Rank, Result, and Actions).
 - * If the race status is pending ("Cho ket qua"), the system allows the spectator to click "Sua" (Edit) or "Xoa" (Delete) to update their prediction.
 - * If the race has ended, the actions are disabled ("Da khoa"), and the system updates the prediction status ("DUNG" / "SAI") along with the spectator's total reward points.

Screen layout:

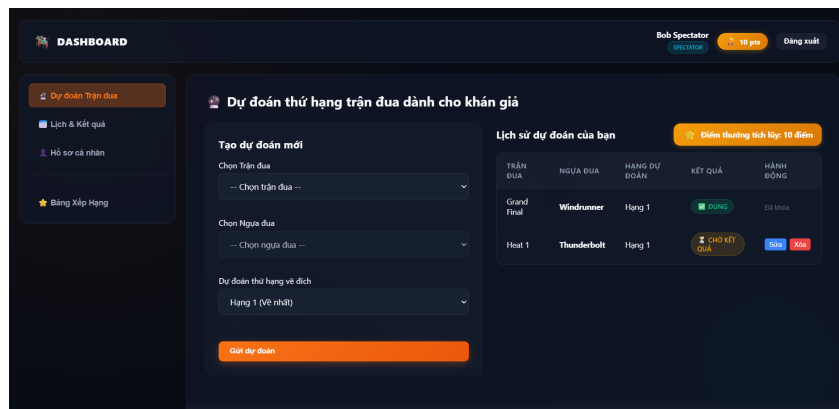


Figure 3.12: Spectator Prediction

3.4.12 View Schedule and Results

Role: Spectator

- **Function trigger:** The spectator navigates to the "Lịch & Kết quả" (Schedule & Results) menu.
- **Function description:**
 - **Purpose:**
 1. Allow the spectator to view the list of upcoming race schedules.
 2. Allow the spectator to view the detailed results and rankings of completed races.
 - **Data processing:**
 1. **For upcoming race schedules ("Lịch thi đấu sắp diễn ra"):**
 - * The system retrieves and displays scheduled races with status "SCHEDULED".
 - * Information includes: Race name (e.g., Heat 1), referee name, date, time, distance, track condition, and the list of participating horses ("Danh sách tham gia") with their respective jockeys.
 2. **For completed race results ("Kết quả các trận đã kết thúc"):**
 - * The system retrieves and displays finished races with status "HOÀN THÀNH".
 - * For each completed race (e.g., Grand Final), the spectator can expand to view the detailed ranking table including: Rank ("Hạng"), Horse ("Ngựa đua"), Jockey, Score ("Điểm"), and Notes ("Ghi chú").

Screen layout:

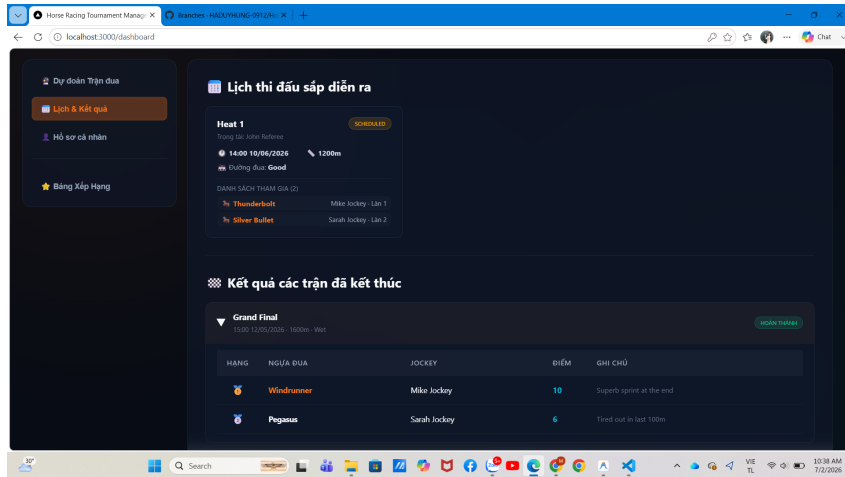


Figure 3.13: View Schedule and Results

3.4.13 Personal Profile

Role: Spectator

- **Function trigger:** The spectator navigates to the "Ho so ca nhan" (Personal Profile) menu.
- **Function description:**
 - **Purpose:**
 1. Allow the spectator to view and update their personal profile information.
 2. Display the spectator's prediction statistics, performance overview, and recent prediction history.
 - **Data processing:**
 1. **For updating profile information:**
 - * The spectator can view and edit the following fields: Full Name ("Ho va Ten"), Email, Phone Number ("So dien thoai"), Avatar URL ("URL Anh dai dien"), Favorite Horse Breed ("Giong ngua yeu thich"), and Favorite Jockey ("Nai ngua yeu thich").
 - * The spectator clicks the "Cap nhat thong tin" button.
 - * The system validates the entered data and updates the profile details in the database.
 2. **For viewing statistics and history:**
 - * The system retrieves and displays statistical insights ("Thong ke & Thanh tich") including: Current Rank ("Thu hang hien tai"), Total Predictions ("Tong so tran du doan"), Accuracy Rate ("Ty le chinh xac"), and Reward Points ("Diem thuong").
 - * The system retrieves and lists the recent prediction history ("Lich su du doan gan day") with columns: Race Name ("Ten tran dua"), Selected Horse ("Ngua da chon"), Time ("Thoi gian"), Status ("Trang thai"), and Reward Points ("Diem thuong").

Screen layout:

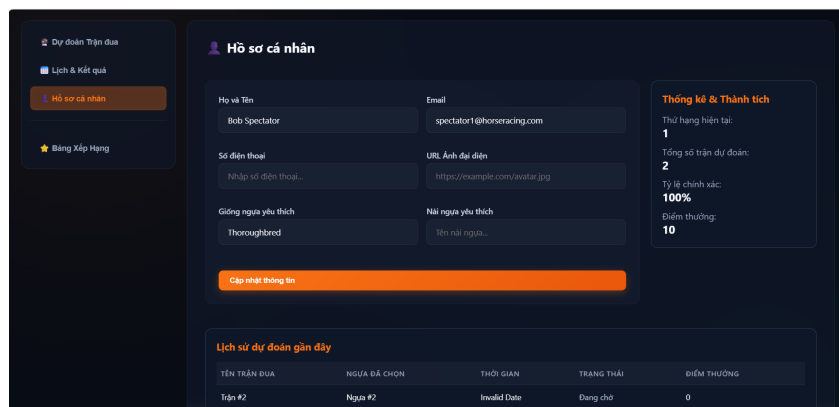


Figure 3.14: View Personal Profile

3.4.14 Leaderboard

Role: Spectator

- **Function trigger:** The spectator navigates to the "Bang Xep Hang" (Leaderboard) menu.
- **Function description:**
 - **Purpose:**
 1. Allow the spectator to view the official competition rankings of horses and jockeys, filtered by tournament.
 2. Allow the spectator to view the top 10 spectators with the highest prediction accuracy scores.
 - **Data processing:**
 1. **For Horse & Jockey Leaderboard ("Ngua & Jockey" tab):**
 - * The spectator can filter rankings by tournament ("Loc theo giai dau") including options: All ("Tat ca"), Summer Championship 2026, or Spring Derby 2026.
 - * The system retrieves and displays the Horse Ranking table ("Xep hang Ngua dua") including columns: Rank ("Hang"), Horse Name ("Ten ngua"), and Accumulated Points ("Diem tích lũy").
 - * The system retrieves and displays the Jockey Ranking table ("Xep hang Jockey") including columns: Rank ("Hang"), Jockey Name ("Ten jockey"), and Accumulated Points ("Diem tích lũy").
 2. **For Top Spectators Leaderboard ("Khan gia xuất sắc" tab):**
 - * The system retrieves and displays the top 10 spectators with the highest prediction scores ("Top 10 Khan gia co diem du doan cao nhất").
 - * The leaderboard table includes columns: Rank ("Hang"), Spectator Name ("Khan gia"), Favorite Horse Breed ("Giống ngựa yêu thích"), and Accumulated Points ("Diem tích lũy").

Screen layout:

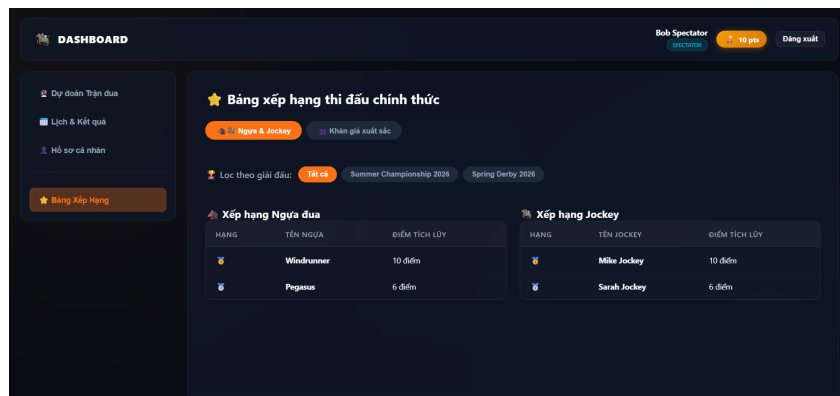


Figure 3.15: Official Competition Leaderboard for Horses and Jockeys

Screen layout:

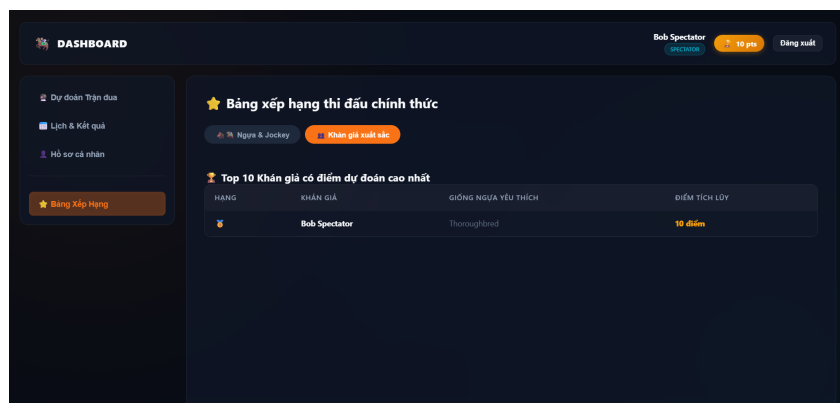


Figure 3.16: View Top 10 Spectators Prediction Leaderboard

3.4.15 Admin Login

Role: Admin

- **Function trigger:** The admin enters account credentials and selects the login button.
- **Function description:**
 - **Purpose:**
 1. Verify the administrator identity.
 2. Provide access to administrative functions.
 - **Data processing:**
 1. The admin enters username and password.
 2. The system validates the credentials.
 3. The system identifies the ADMIN role.
 4. The system creates an admin session.

- **Function details:**

- Successful login redirects to the Admin Dashboard.
- Invalid credentials show an error message.
- Admin can access system management screens after login.

Screen layout:

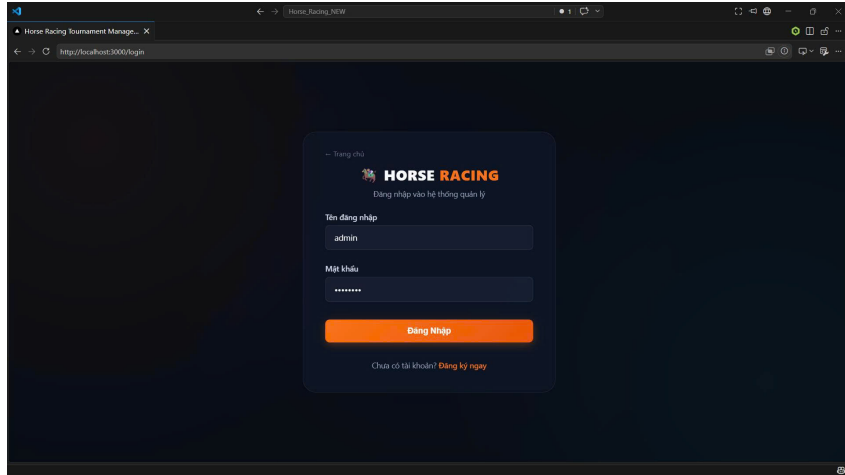


Figure 3.17: Admin Login

3.4.16 System Overview

Role: Admin

- **Function trigger:** Select "Tong quan he thong" from the admin menu.
- **Function description:**
 - **Purpose:**
 1. Provide a summary of system status.
 2. Display key metrics and role distributions.
 - **Data processing:**
 1. Retrieve user counts by role.
 2. Retrieve tournament and race statistics.
 3. Retrieve award and registration metrics.
- **Function details:**
 - Display total active users, tournaments, races, horses, and awards.
 - Show distribution of users by role.
 - Show tournament status and prediction statistics.

Screen layout:

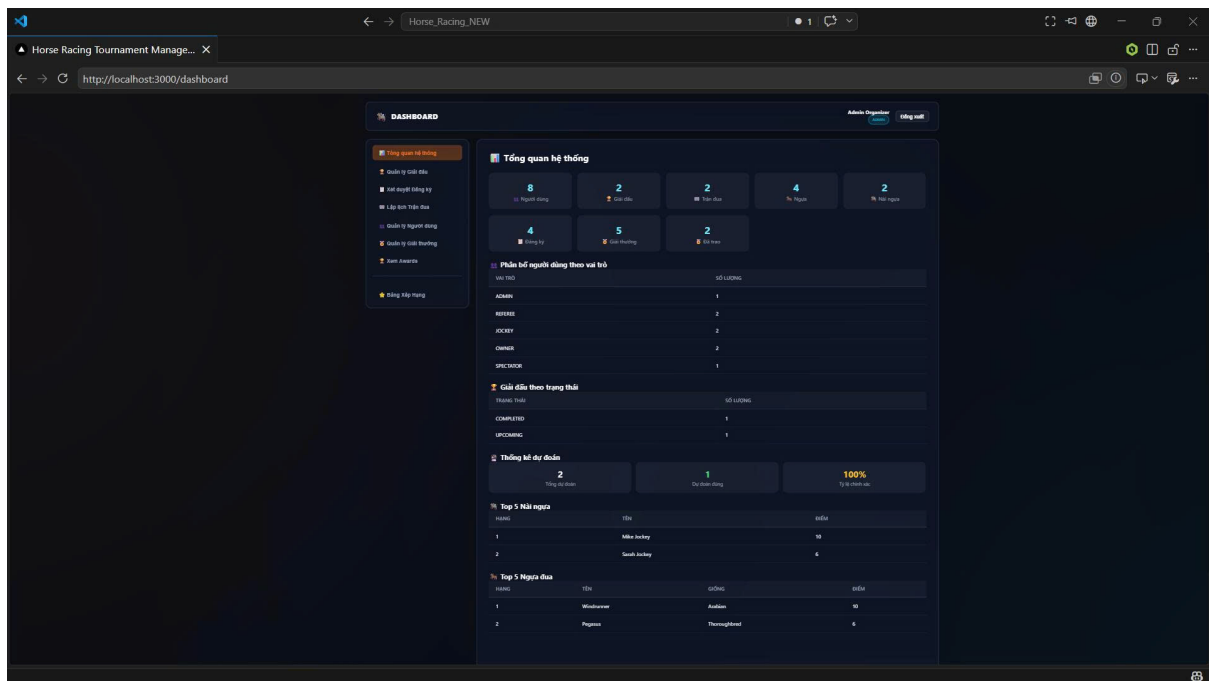


Figure 3.18: System Overview

3.4.17 Manage Tournaments

Role: Admin

- **Function trigger:** Select "Quan ly Giai dau".
- **Function description:**
 - **Purpose:**
 1. Create and manage tournament information.
 2. Add new rounds to tournaments.
 - **Data processing:**
 1. Enter tournament name, description, dates, and location.
 2. Save tournament details.
 3. Add competition rounds with sequence order.
- **Function details:**
 - Create new tournaments.
 - Edit or delete tournaments.
 - Manage the number and order of rounds.
 - Display current tournament list and status.

Screen layout:

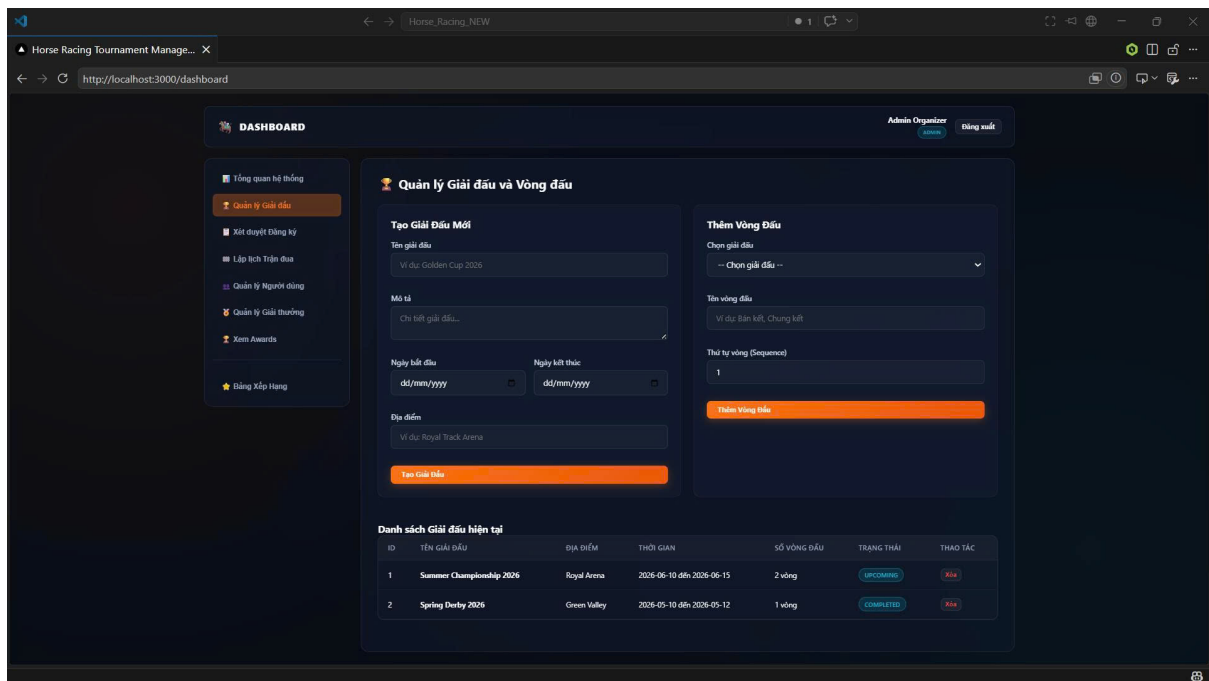


Figure 3.19: Manage Tournaments

3.4.18 Schedule Races

Role: Admin

- **Function trigger:** Select "Lap lịch Trộn đua".
- **Function description:**
 - **Purpose:**
 1. Schedule race heats and assign race details.
 2. Organize race lanes and pairs.
 - **Data processing:**
 1. Choose the tournament round.
 2. Enter race name, date, distance, and conditions.
 3. Select approved horse-jockey pairs.
 4. Assign lane numbers for each pair.
- **Function details:**
 - Create and manage races.
 - Assign horses and jockeys to lanes.
 - Display current race schedule and status.

Screen layout:

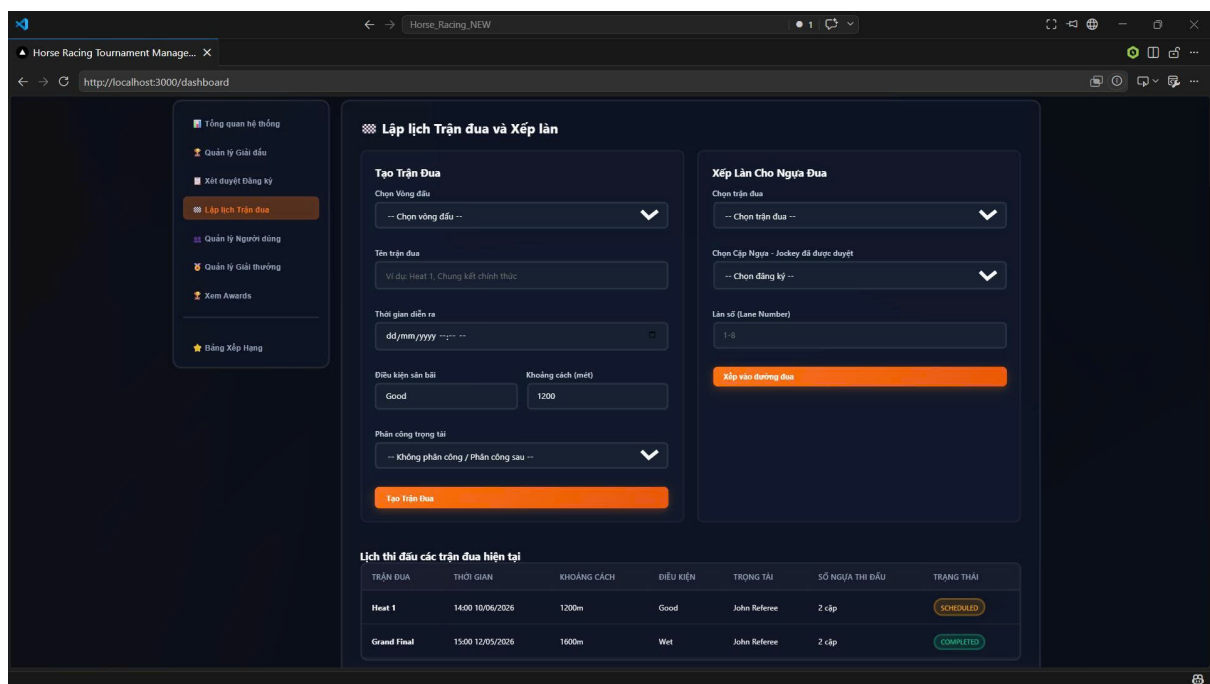


Figure 3.20: Schedule Races

3.4.19 Manage Rewards

Role: Admin

- **Function trigger:** Select "Quan ly Giai thuong".
- **Function description:**
 - **Purpose:**
 1. Define prizes for tournament positions.
 2. Manage award distributions.
 - **Data processing:**
 1. Select the tournament.
 2. Enter prize rank, title, value, and description.
 3. Save reward information.
- **Function details:**
 - Add new awards for tournaments.
 - Edit or delete existing awards.
 - Display award list and delivery status.

Screen layout:

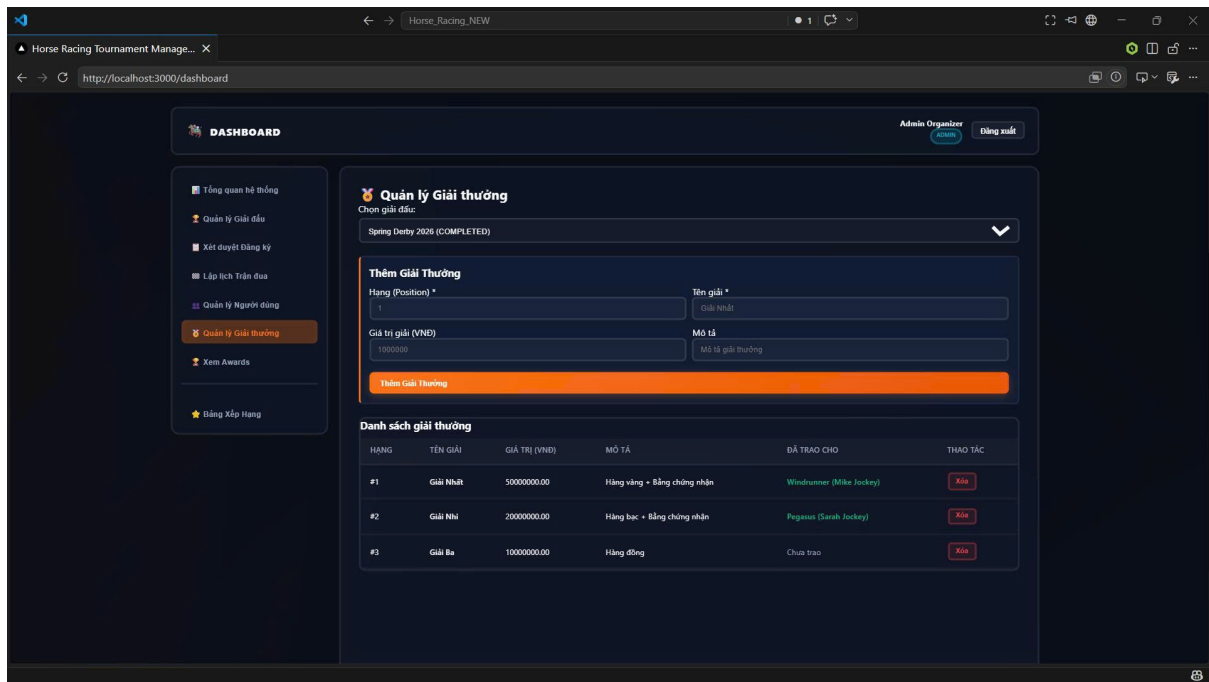


Figure 3.21: Manage Rewards

3.4.20 User Management

Role: Admin

- **Function trigger:** Select "Quan ly Nguoi dung".
- **Function description:**
 - **Purpose:**
 1. Manage system users and roles.
 2. Control access and account status.
 - **Data processing:**
 1. Retrieve user accounts.
 2. Display role and status information.
 3. Perform account lock or unlock actions.
- **Function details:**
 - Display user list with roles and statuses.
 - Lock or unlock user accounts.
 - Filter users by role.

Screen layout:

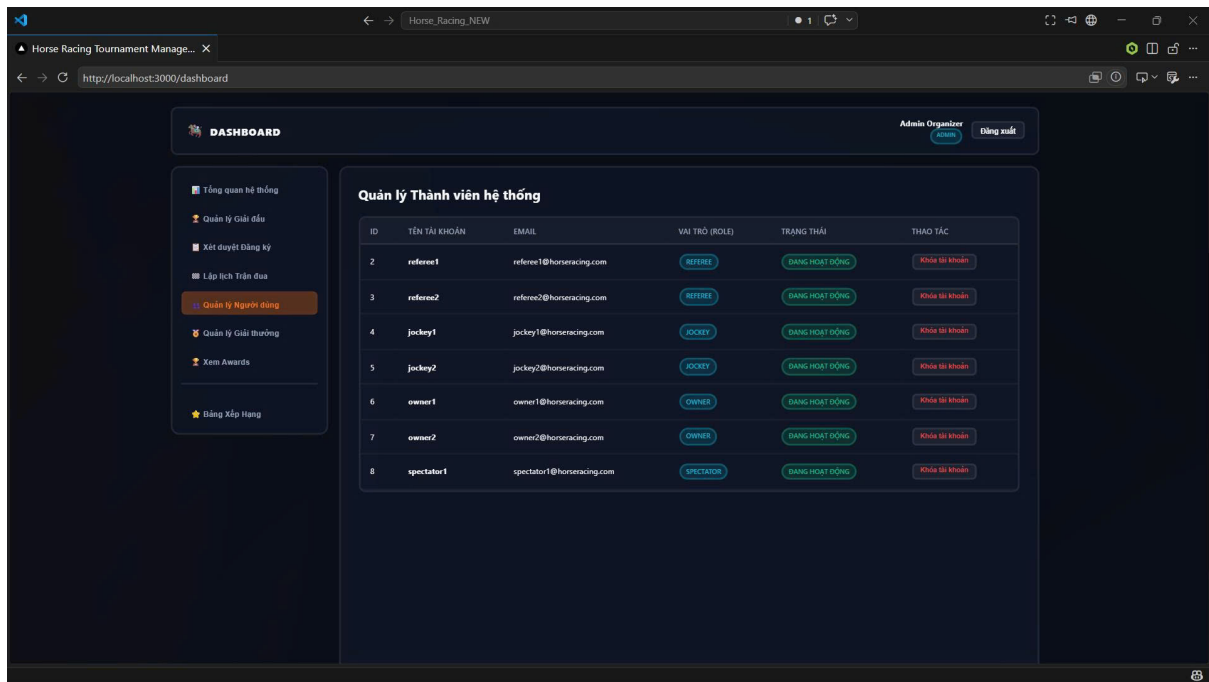


Figure 3.22: User Management

3.4.21 Confirm Registrations

Role: Admin

- **Function trigger:** Select "Xet duyet Dang ky".
- **Function description:**
 - **Purpose:**
 1. Review and confirm registration requests.
 2. Approve or reject tournament entries.
 - **Data processing:**
 1. Retrieve pending registrations.
 2. Display horse, jockey, and tournament details.
 3. Update registration approval status.
- **Function details:**
 - Show pending registration entries.
 - Approve or reject registrations.
 - Display current approval status.

Screen layout:

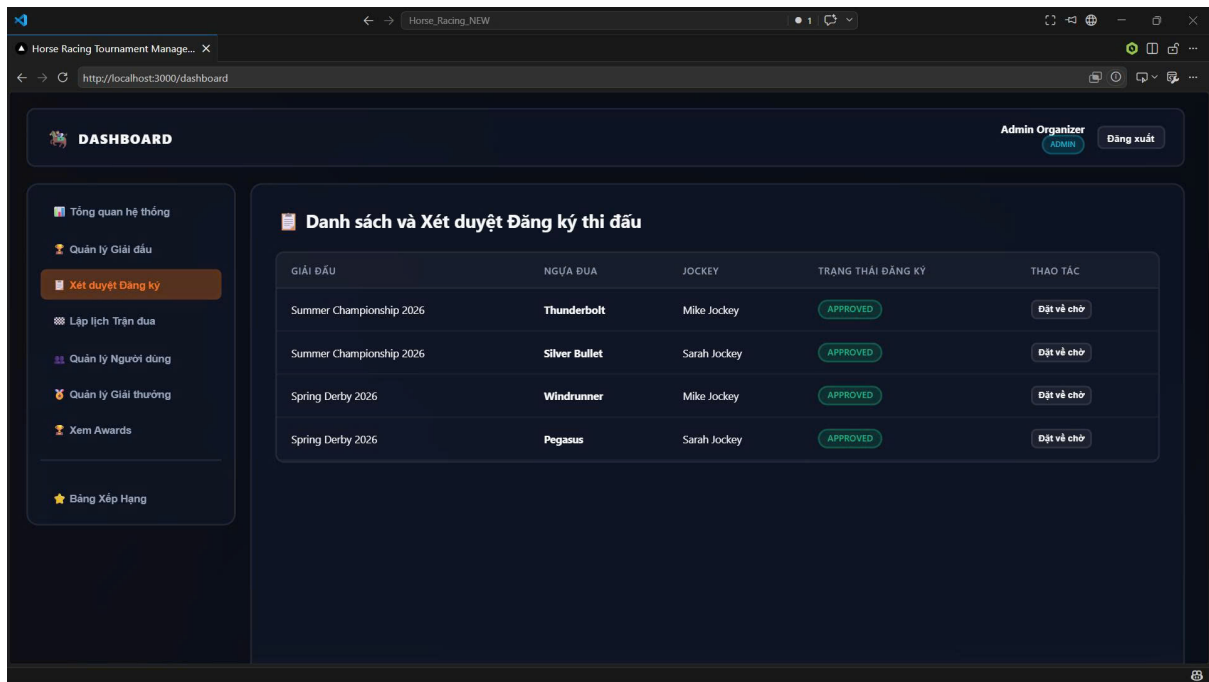


Figure 3.23: Confirm Registrations

3.4.22 Leaderboard

Role: Admin

- **Function trigger:** Select "Xem Awards" or "Bang Xep Hang".
- **Function description:**
 - **Purpose:**
 1. Display the official leaderboard.
 2. Review award winners and rankings.
 - **Data processing:**
 1. Retrieve ranking data for horses and jockeys.
 2. Retrieve award recipient details.
- **Function details:**
 - Display award and ranking summaries.
 - Show top positions by horse and jockey performance.

Screen layout:

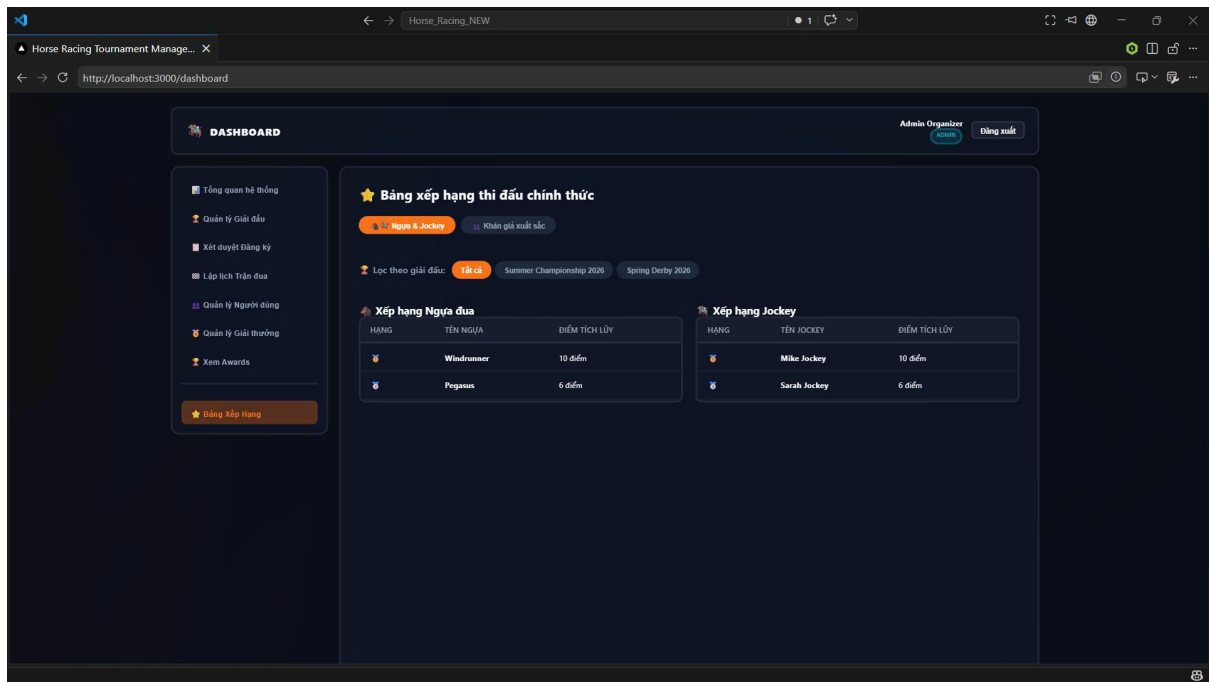


Figure 3.24: Leaderboard

3.5. Non-Functional Requirements

3.5.1 External Interfaces

To ensure effective communication between users and external systems, the following interfaces shall be supported.

User Interface

- The system shall provide a user-friendly and intuitive interface for Horse Owners, Jockeys, Referees, and Administrators.
- The graphical interface shall follow modern web design standards and support responsive layouts.
- Users shall be able to access the system through desktop computers, tablets, and mobile devices.
- Navigation menus and functions shall remain consistent throughout the system.

Authentication Interface

- The system shall support secure authentication using username and password.
- Role-based access control shall restrict system functions according to user roles.
- User sessions shall be managed securely to prevent unauthorized access.

Database Interface

- The application shall communicate with the database system to store and retrieve information.
- Data synchronization between the application server and database server shall be maintained.
- The database shall support CRUD operations for horses, tournaments, registrations, invitations, and users.

3.5.2 Quality Attributes

Usability

User-Friendly Interface

- The system shall provide a simple and intuitive interface.
- Functions such as tournament registration and jockey invitation shall require minimal user actions.
- The interface shall be easy to understand for both novice and experienced users.

Training and Familiarization Time

- Horse Owners and Jockeys shall require less than 30 minutes to learn the basic functions.
- Administrators shall require approximately 1–2 hours to understand management operations.

Usability Metrics

- Tournament registration shall be completed within 3 minutes.
- Error messages shall clearly guide users to correct invalid input.
- User feedback shall be collected to improve usability.

Reliability

Availability

- The system shall maintain at least 99% availability.
- Maintenance shall be scheduled during low-traffic periods.

Data Integrity

- Registration and tournament information shall be stored accurately.
- Duplicate registrations shall be prevented.

- Data consistency shall be maintained across all modules.

Error Handling

- The system shall display meaningful error messages.
- Critical system errors shall be logged for administrators.
- The system shall recover from failures with minimal data loss.

Performance

Response Time

- Normal user requests shall respond within 2 seconds.
- Under heavy load conditions, response time shall not exceed 5 seconds.

Concurrent Users

- The system shall support at least 500 concurrent users.
- Multiple users shall access the system simultaneously without significant performance degradation.

Database Performance

- Search operations shall complete within 3 seconds.
- Registration transactions shall be processed efficiently.

Security

- Passwords shall be encrypted before storage.
- Unauthorized access attempts shall be logged.
- Session timeout mechanisms shall automatically log out inactive users.
- Input validation shall prevent invalid data submission.

Maintainability

- The system shall follow a modular architecture.
- Source code shall comply with coding standards.
- New features shall be added with minimal impact on existing modules.
- API endpoints and database structures shall be documented.

Portability

- The system shall support Google Chrome, Microsoft Edge, and Mozilla Firefox.
- The application shall operate on both Windows and Linux servers.
- Users shall access the system without additional software installation.

IV. Software Design Description

4.1. Overall Design

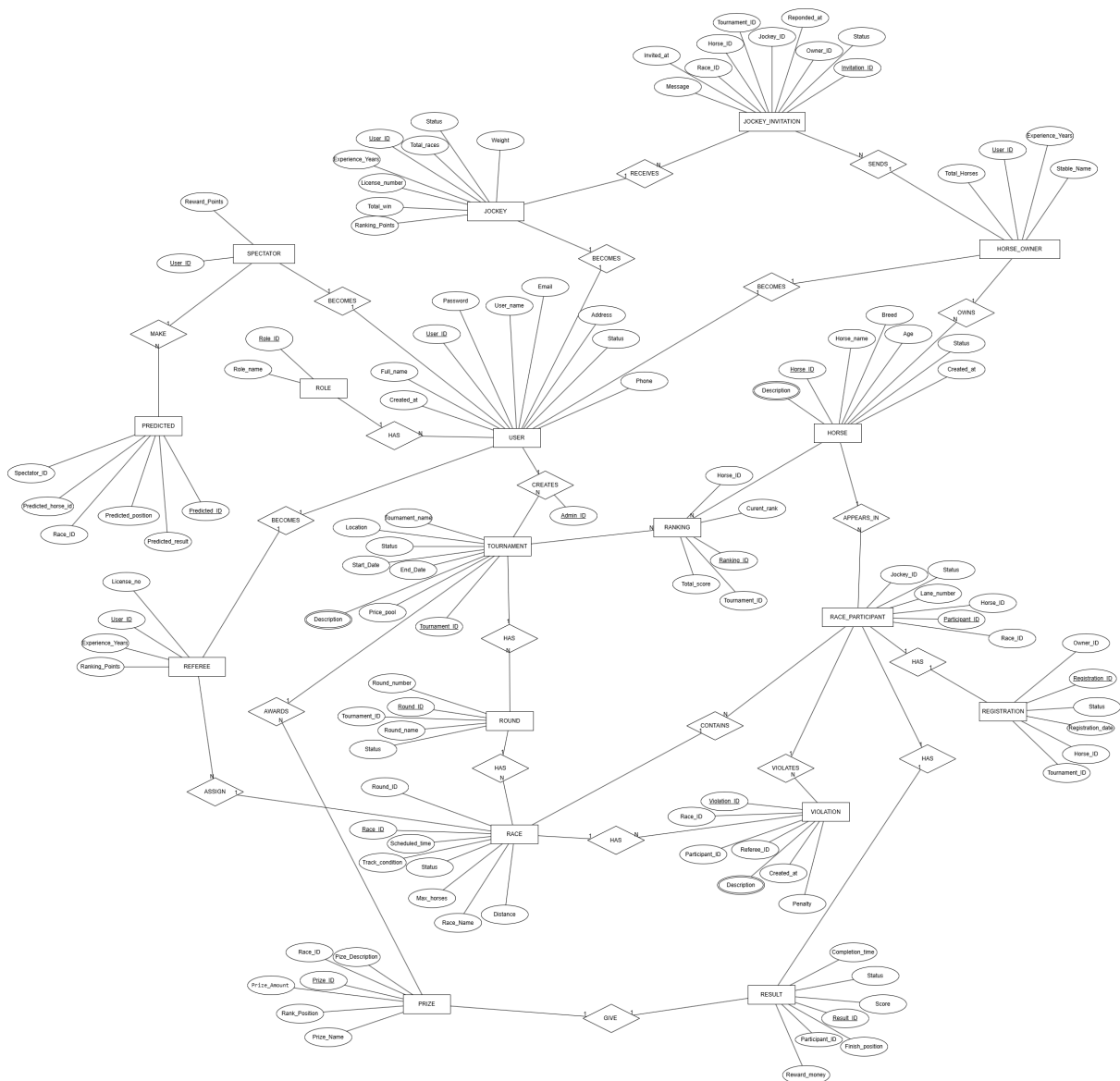


Figure 4.1: Entity Relationship Diagram

4.2. Detailed Design

4.2.1 Admin

4.2.2 Horse Owner Login and Registration

Sequence Diagram

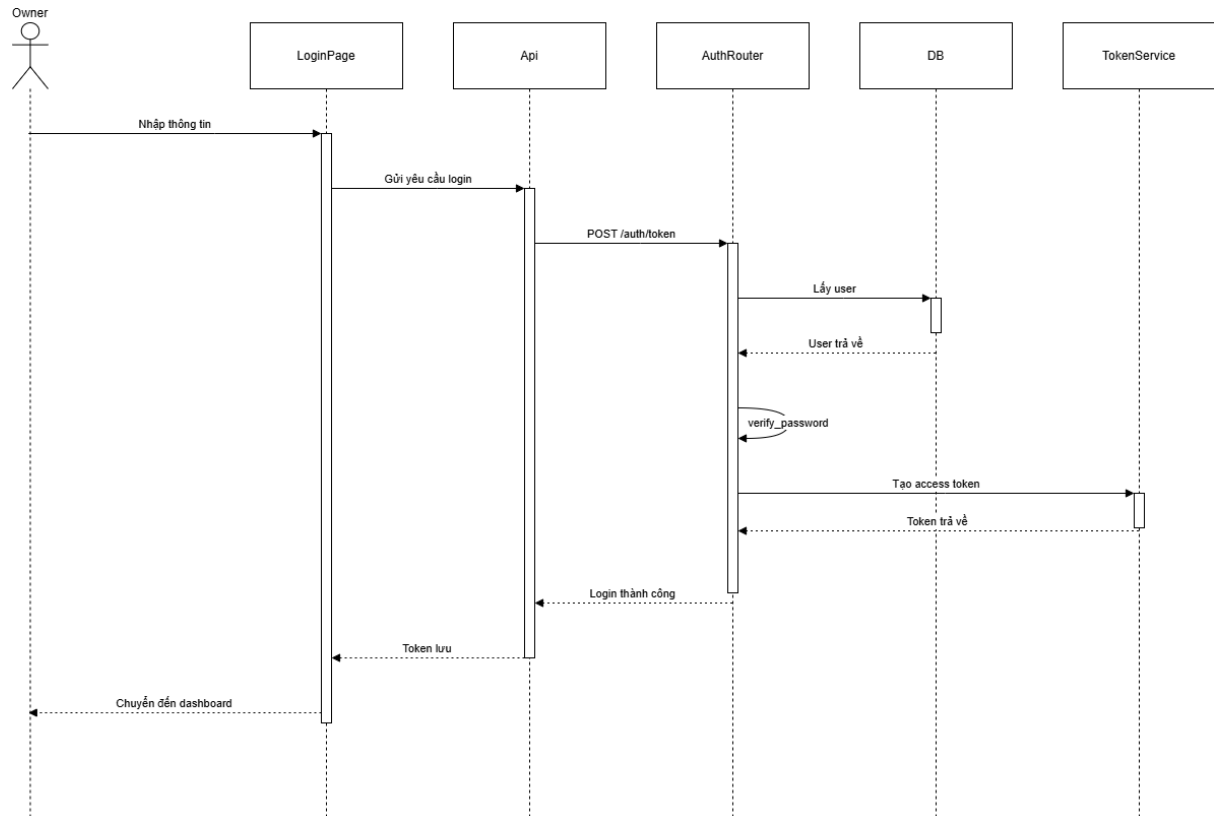


Figure 4.2: Horse Owner Login Sequence Diagram

Class Diagram

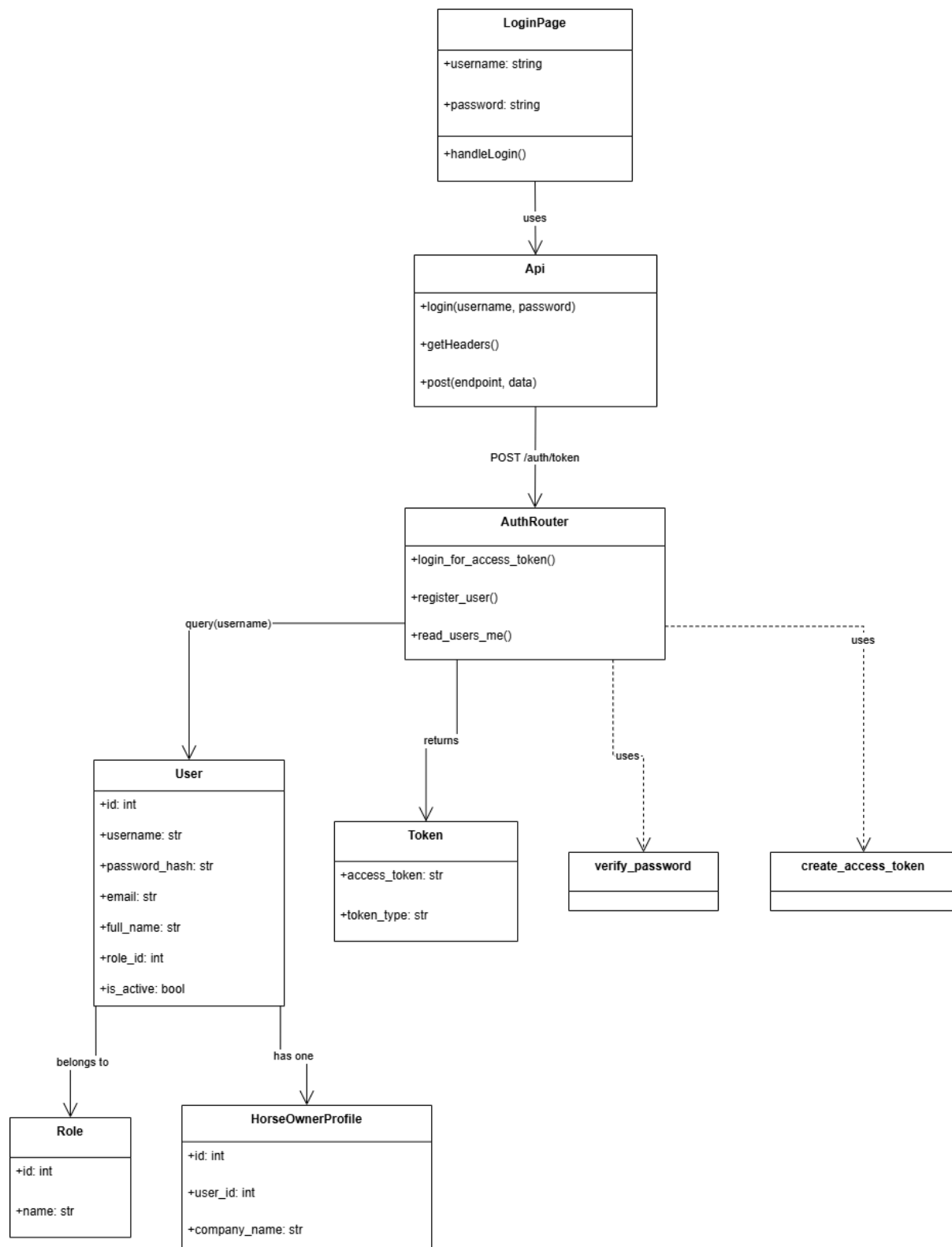


Figure 4.3: Horse Owner Login Class Diagram

Class Specification

Class	Attributes / Methods	Type	Description
LoginPage	username, password, handleLogin()	state / action	UI component that collects login credentials and calls the API.
Api	login(), post(), getHeaders()	service / helper	Calls POST /auth/token , stores JWT, and prepares request headers.
AuthRouter	login_for_access_token(), read_users_me()	controller / route	Validates username/password and issues bearer tokens.
User	id, username, password_hash, email, full_name, role_id, is_active	entity	Database entity representing an application user.
Role	id, name	entity	Defines user roles, including OWNER .
Token	access_token, token_type	DTO	Response payload returned after successful login.
HorseOwner Profile	id, user_id, company_name	entity	Owner-specific profile linked to the User entity.

Table 4.1: Horse Owner Login class specification

4.2.3 Jockey

4.2.4 Spectator

4.2.5 Referee